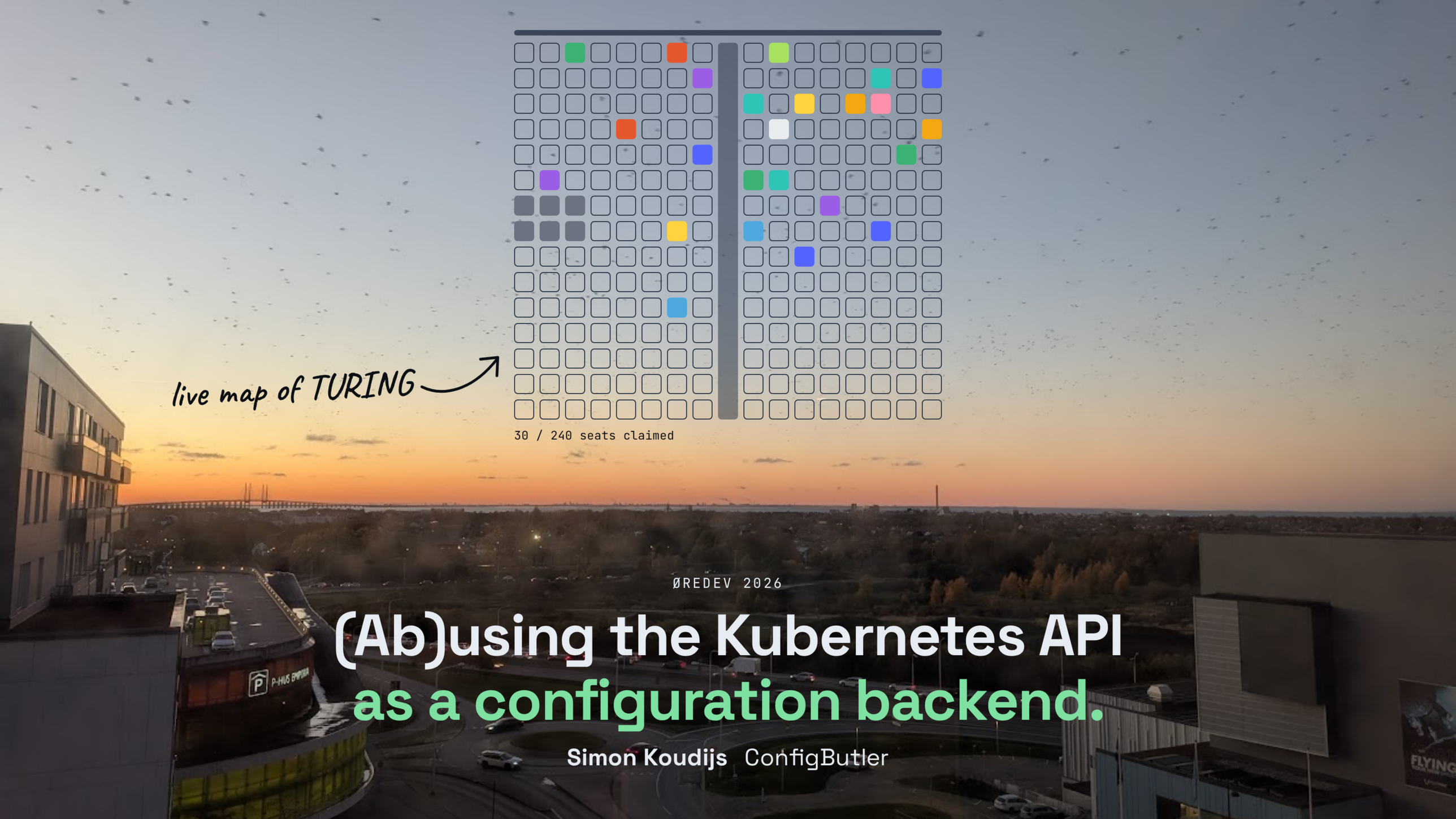


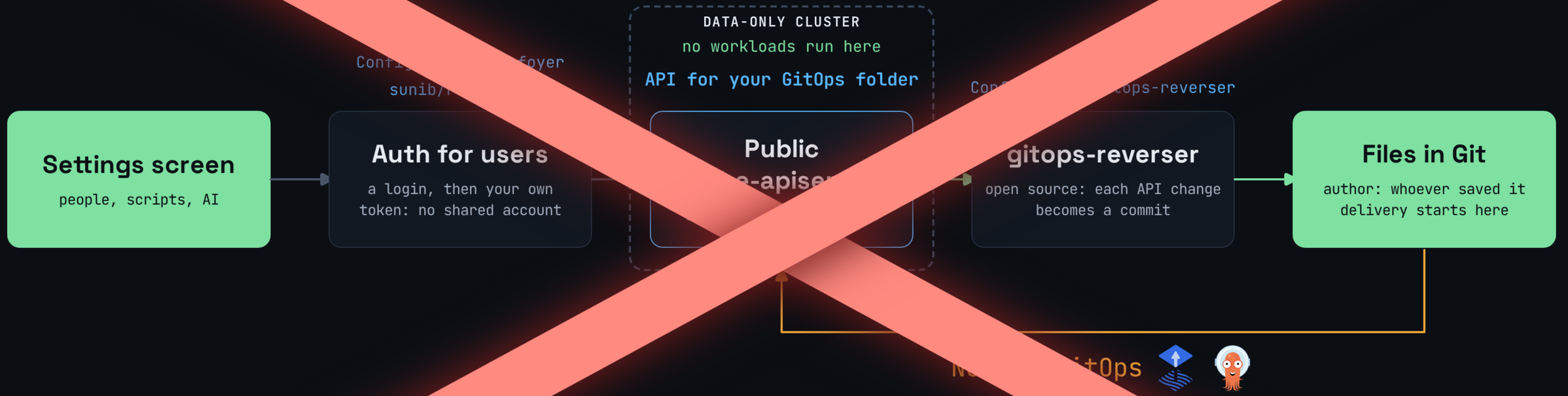
ØREDEV 2026

(Ab)using the Kubernetes API as a configuration backend.

Simon Koudijs ConfigButler

As an industry, we should be much better at settings.

An … nd files. At the same time.



KR… it possible: one shape, in the API and … files

Today I'll show the trick.

This is not an infra talk.

TLDR;

A control plane, just for your settings.

1. **It's just an API**
   kube-apiserver and etcd, nothing else. One shape for API and file, and types you define.

2. **Identity is kept**
   Your own login, for people and workloads, all the way to the commit.

3. **The API forgets, Git remembers**
   GitOps for configuration: every change a commit, with who and why.

4. **It doesn't always fit**
   The downsides, and practical criteria for when to use it.

And I'll tell you what I don't know yet.



Simon Koudijs

`[Software|Product|Cloud|AI] engineer`

- From the Netherlands
- Left my engineering job in 2025 to build my own company
- Frustrated by settings pages: do we still build that ourselves?
- **koudijs.dev**: consultancy, training, speaking
- **ConfigButler**: the hosted product of this idea

YOU ANSWERED THIS AT THE DOOR

What is a CRD?



27 answered

YESTERDAY'S TALK, IN ONE SLIDE

Yesterday, your workload only read.
Today, it also gives back.

YESTERDAY



Settings ship with your workload, promoted together. **The workload never talks to Kubernetes.**

TODAY



Your workload gets direct access. **It reads, and it reports back.**

YESTERDAY

The slides of both talks
koudijs.dev/speaking

THE PROVIDER

What do API providers expect these days?

Tooling support
CLIs and GUIs that just work

Help with AI
A format that agents already know

Conflict resolution
Two edits at once, without losing one

Live watches
Push the change, no polling

Great RBAC
Who may change what

Audit log
Who did what, and when

And I don't want to build all of that myself.

PART 1 OF 4

Drop the infra. Keep the API.

1 **It’s just an API**
kube-apiserver and etcd, nothing else. One shape for API and file, and types you define.

2 **Identity is kept**
Your own login, for people and workloads, all the way to the commit.

3 **The API forgets, Git remembers**
GitOps for configuration: every change a commit, with who and why.

4 **It doesn’t always fit**
The downsides, and practical criteria for when to use it.

That chair you just coloured
is a Kubernetes object.

WHAT'S BEHIND THAT URL

It's just an API.

```
GET https://demo.koudijs.dev/k8s/apis/examples.configbutler.ai/v1alpha1
 /namespaces/voter/seats/ordev-2-turing-7-1
```

k8s? That should be settings or config.

A single cluster, just for this

kube-apiserver
the API

⇄

etcd
where it's stored

for example  kcp  Kamaji

Your infra clusters

Your pods and nodes. **No end users in here.**

A single cluster for this. Don't let end users poke in your infra.

Is this a database,
or a folder of files?

One object says everything about itself.

```yaml
apiVersion: examples.configbutler.ai/v1alpha1
kind: Seat

metadata:
  namespace: voter
  name: ordev-2-turing-7-1

spec:
  color: '#a8e05f'
  emoji: 🐧

status:
  lastCommit:
    message: 'Green for the team'
```

kind

What is it?

A type with a schema

metadata

Which one?

Its identity, inside the object

spec

What do you want?

The desired state

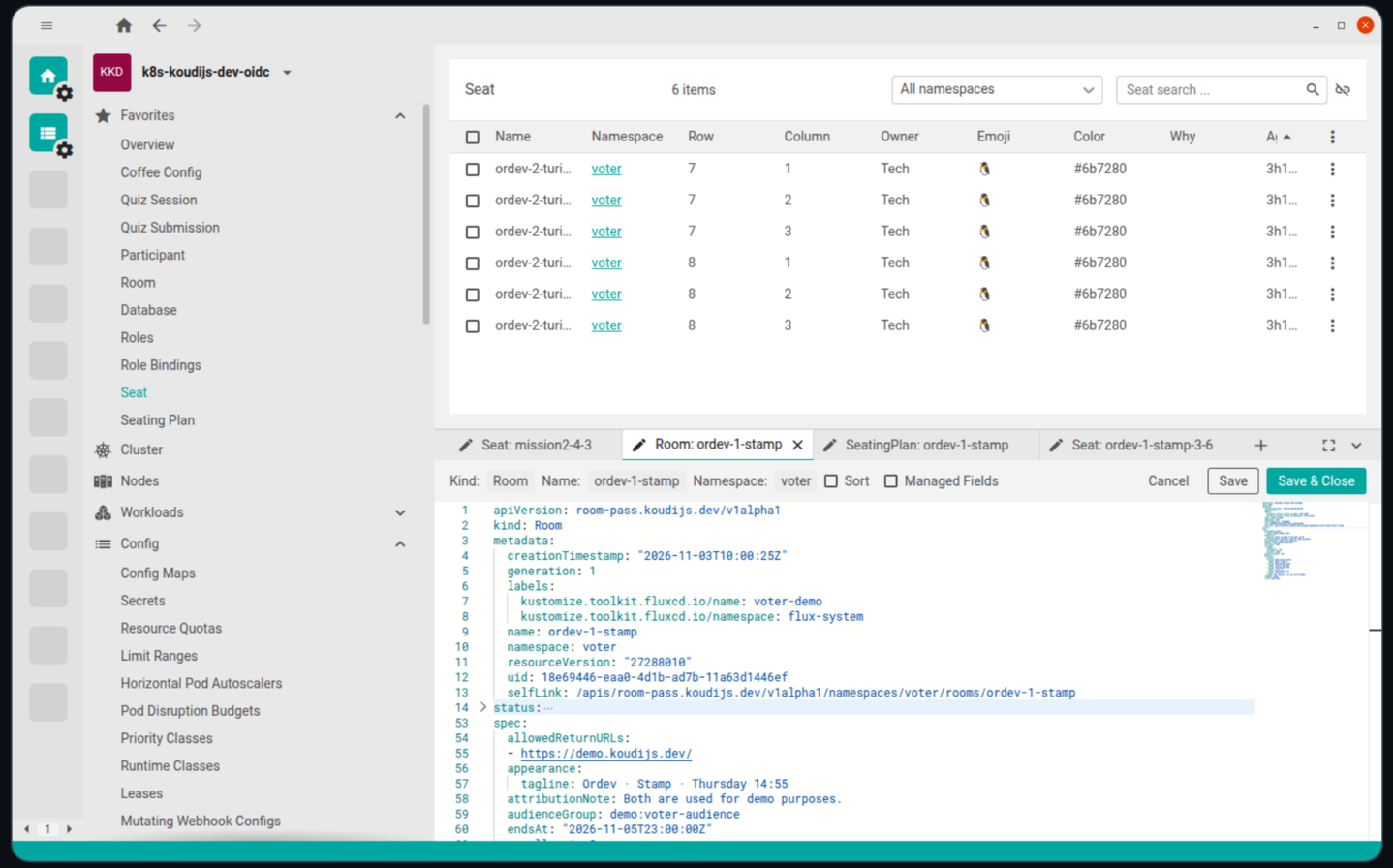
status

What happened?

From the control loop. Not in Git

```
.../apis/examples.configbutler.ai/v1alpha1/namespaces/voter/seats/ordev-2-turing-7-1
```

One schema. Many instances.



Custom Resource

an instance

One chair: `ordev-2-turing-7-1`

Consumer fills it in, using the app

Custom Resource Definition

the interface, the schema

What every `Seat` must look like: a row, a colour, an emoji

Provider defines it, during application development

Freelens knows nothing about chairs. It reads the CRD.

THE REAL ONE, SHORTENED

The definition of your chair: also just YAML.

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: seats.examples.configbutler.ai
spec:
  group: examples.configbutler.ai
  names: { kind: Seat, plural: seats }
  versions:
    - name: v1alpha1
      schema:
        openAPIV3Schema:
          properties:
            spec:
              properties:
                row:
                  type: integer
                  x-kubernetes-validations:
                    - rule: self == oldSelf
                      message: 'A seat does not move: row is immutable.'
                color: { type: string, pattern: '^#[0-9a-fA-F]{6}$' }
                emoji: { type: string, enum: [🦊, 🐙, 🦄, …] }
```

- **It's an object too.** Apply it, and the API speaks `Seat`
- **It names the type.** That's where `/seats` in the URL comes from
- **It says what's valid.** A pattern for the colour, and 24 emoji that read well on a projector
- **Rules live here too.** A CEL rule makes row, column and owner immutable. No webhook, no server to run

The provider writes this. The consumer fills it in.

What the consumer sees: just your chair.

```yaml
apiVersion: examples.configbutler.ai/v1alpha1
kind: Seat
metadata:
  name: ordev-2-turing-7-1  # plan-row-column
spec:
  seatingPlanRef: { name: ordev-2-turing }
  row: 7                    # immutable
  column: 1                 # immutable
  claimedBy:                # immutable
    displayName: simon-on-stage
    username: demo:Cg5zaW1v…
  color: '#a8e05f'          # may change
  emoji: 🐧                 # one of 24
```

- **The seat is in the name.** Two phones claim one seat: the second gets `409 AlreadyExists`. Names are unique, so the first claim wins
- **Spec is the intent.** That is what goes to Git; secrets go in their own Secret

It's real design work. But it's **your app's model**, not another settings table.

OPEN SOURCE

ConfigButler skills
github.com/ConfigButler/skills
homemade, for you and your AI agents

TWO EDITS AT ONCE

Last writer wins, per field.

Alice saves

```
spec:
  color: '#a8e05f'
```

+

Bob saves

```
spec:
  emoji: 🐧
```

=

The object

```
spec:
  color: '#a8e05f'
  emoji: 🐧
```

For editing, that's enough. You only send the fields you changed

A watch keeps everyone current. Your screen shows the other edit right away

Need it stricter? Send the `resourceVersion` and get a `409`, like my coffee editor

My advice: make your edits through the API.

FROM THE PEOPLE WHO BUILT THE KUBERNETES API

How they designed it.

“declarative abstractions are primary, rather than layered on top of the system”

Say what you want. That’s the foundation.

“no hidden internal resource-oriented APIs”

Your code gets the same API as Kubernetes’ own.

“designed for humans to directly author and read”

No second format to keep in sync.

“Comprehensive version control is the responsibility of other systems (e.g., git).”

READ IT

The Kubernetes Resource Model, Brian Grant, 2018

github.com/kubernetes/design-proposals-archive/blob/main/architecture/resource-management.md

The control loop: it never stops checking.



The provider is the loop. The consumer can read it.

WE'RE IN THE ARCHITECTURE TRACK

Good design is still SOLID.

S — **Single responsibility**

A data-only control plane. It stores and serves your settings, and nothing else.

O — **Open/closed**

Add your own types with a CRD. The API server itself doesn't change.

D — **Dependency inversion**

You define your interface: the CRD. The config control plane does the editing, and gets you a concrete implementation: the CR.

Think of it as DI for config on steroids.

Would a hosted one help you?
Or would you just run it yourself?

PART 2 OF 4

Who are you? This is where it shines.

1 **It’s just an API**
kube-apiserver and etcd, nothing else. One shape for API and file, and types you define.

2 **Identity is kept**
Your own login, for people and workloads, all the way to the commit.

3 **The API forgets, Git remembers**
GitOps for configuration: every change a commit, with who and why.

4 **It doesn’t always fit**
The downsides, and practical criteria for when to use it.

My token can read. It can’t delete.

```
$ export CURL_CA_BUNDLE=~/.kube/koudijs-ca.crt

$ BASE=https://api.k8s.koudijs.dev:6443
G=examples.configbutler.ai
V=v1alpha1
R=seats

$ SEAT=$BASE/apis/$G/$V/namespaces/voter/$R/ordev-2-turing-7-1

$ kubectl describe role seat-reader -n voter

$ TOKEN=$(kubectl create token seat-reader -n voter --duration=1h)

$ echo $TOKEN | jq -R 'split(".")[1] | @base64d | fromjson' \
  | jq '{who: .sub, expires: (.exp | todate)}'

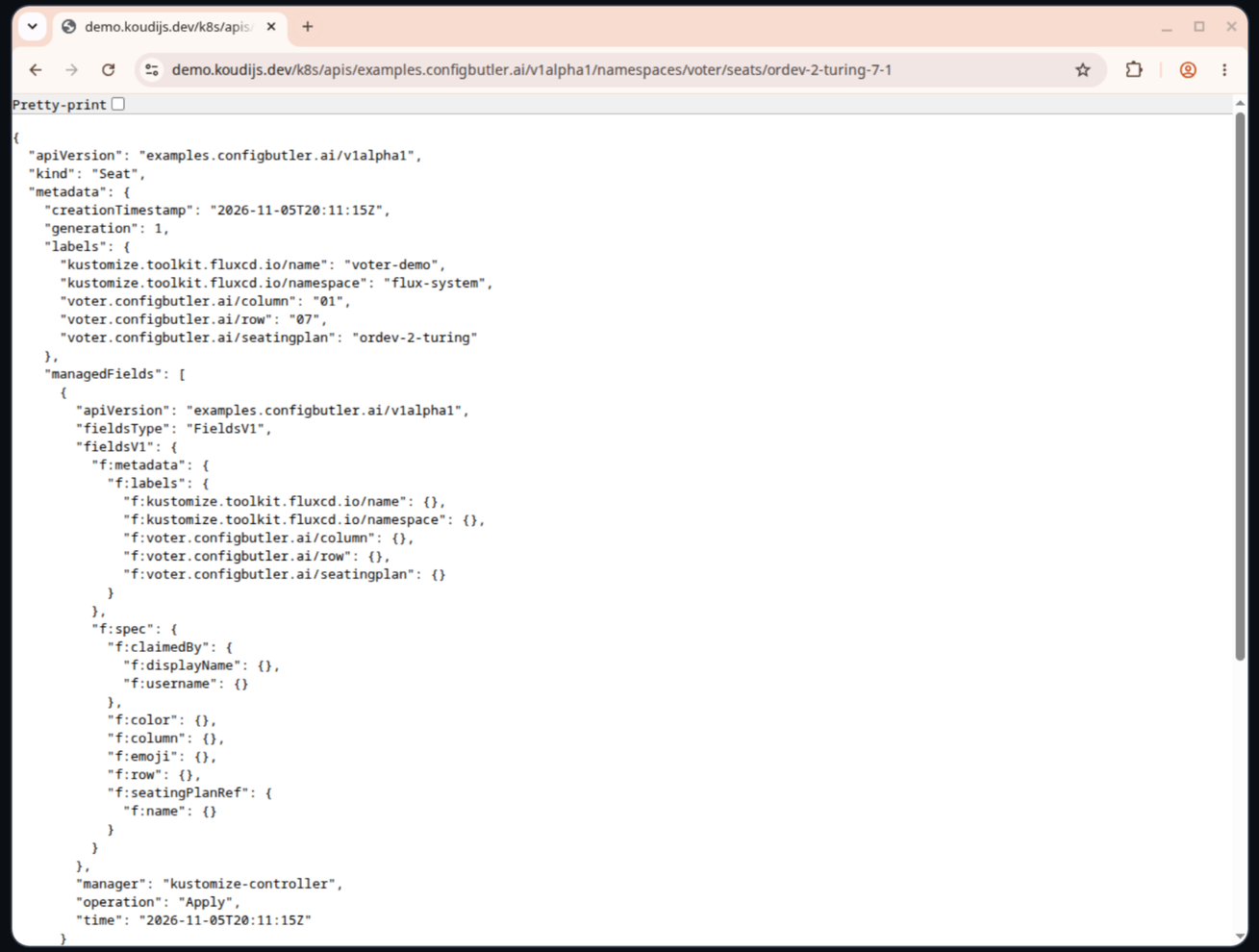
$ curl -s -H "Authorization: Bearer $TOKEN" $SEAT \
  | jq '{kind, spec}'

$ curl -s -X DELETE -H "Authorization: Bearer $TOKEN" $SEAT \
  | jq .message
```

krm-foyer github.com/ConfigButler/krm-foyer **Room Pass** github.com/sunib/room-pass

THE SAME API, IN A BROWSER

Machines get a token. People just log in.



Room Pass

A room code and a name: no accounts. This is how you got in today

krm-foyer

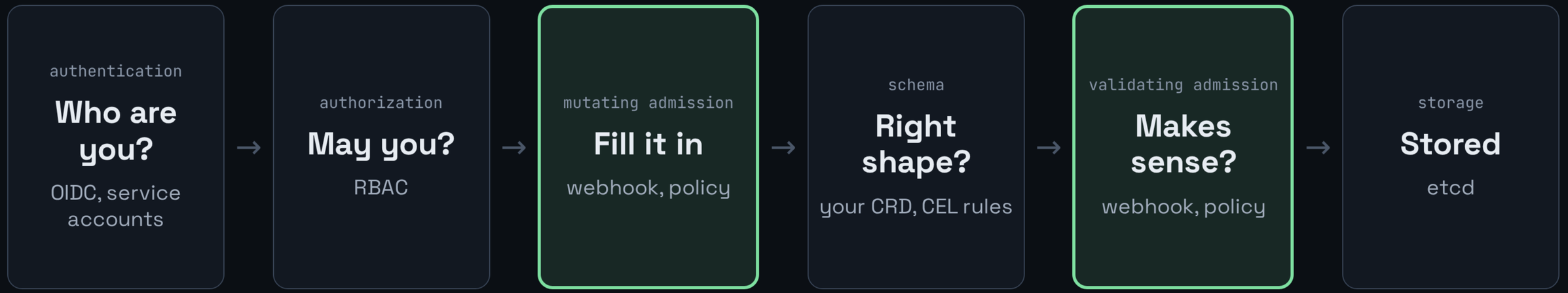
Login and session on your app's own domain: a same-origin cookie, no CORS. Forwards `/k8s`, or `/settings`, or whatever you like

kube-apiserver

Still decides everything. Neither of the two decides what you may do

ONE REQUEST THROUGH THE KUBE-APISERVER

You get to plug in here.



Audit: every request, with who you are

Open/closed again: you don’t change the pipeline, you plug in.

POLICY, FOR THE COFFEE WEBSHOP

Nobody pays more than €10 for a coffee.

```yaml
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: voter-coffee-price
spec:
  matchConstraints:
    resourceRules:
      - apiGroups: [examples.configbutler.ai]
        apiVersions: ['*']
        resources: [coffeeconfigs]
        operations: [CREATE, UPDATE]
  validations:
    - expression: >-
        object.spec.products.all(p,
          p.priceCents <= 1000)
      message: Nobody pays more than €10 for a coffee
```

- **A rule about the data.** RBAC says if you may edit the menu. This says if the edit makes sense
- **Inside the API server.** A CEL expression: no webhook, and no server to run
- **For everyone.** A €12 coffee from Git is refused too

422

Nobody pays more than €10 for a coffee: Cappuccino.

PART 3 OF 4

The API forgets.

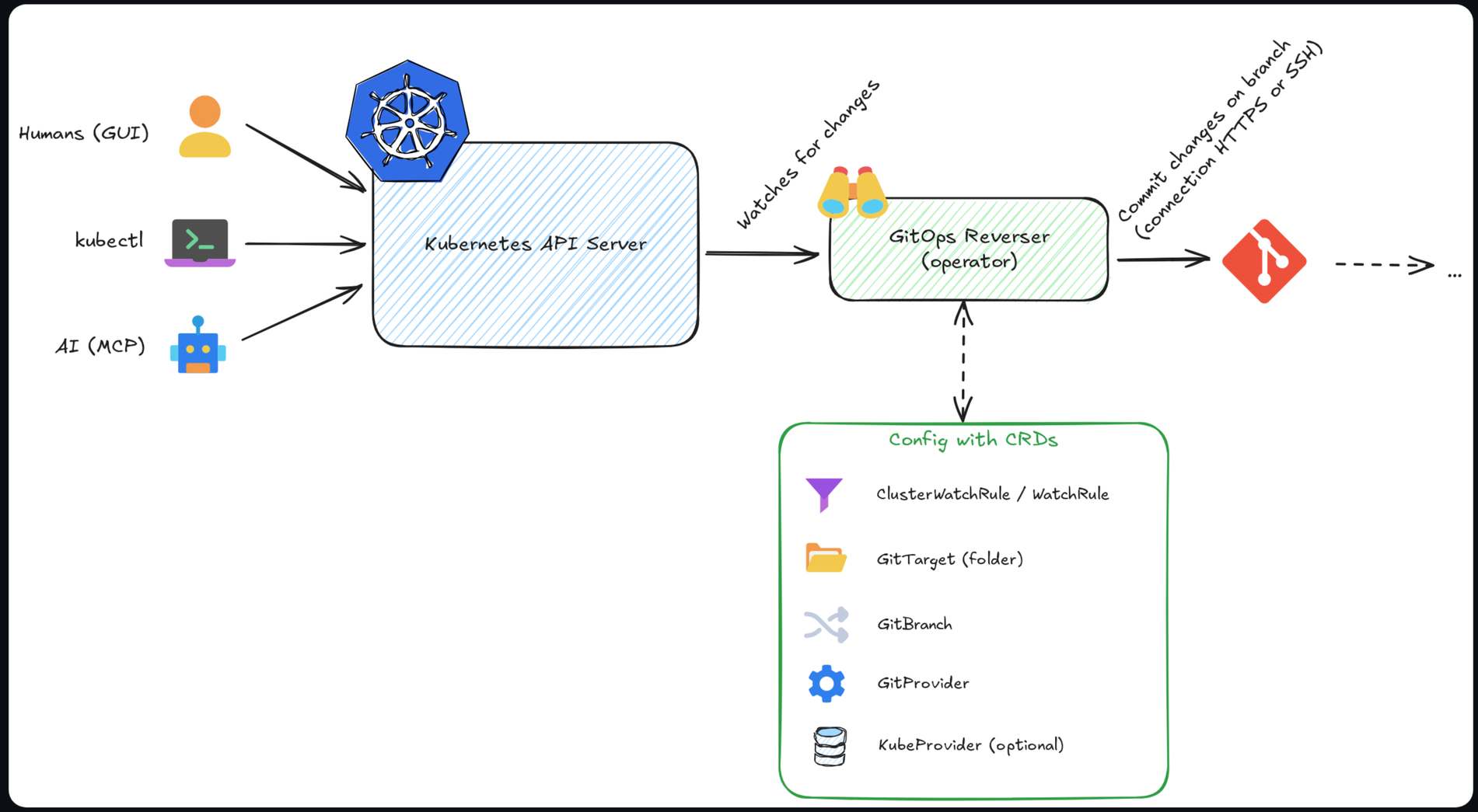
1 **It’s just an API**
kube-apiserver and etcd, nothing else. One shape for API and file, and types you define.

2 **Identity is kept**
Your own login, for people and workloads, all the way to the commit.

3 **The API forgets, Git remembers**
GitOps for configuration: every change a commit, with who and why.

4 **It doesn’t always fit**
The downsides, and practical criteria for when to use it.

Every change becomes a commit.



With the name of whoever saved it.

BI-DIRECTIONAL GITOPS

One document. Both directions.



Nothing to translate. It's the same object on both sides, so nothing gets lost on the way

Edits: through the API. It checks them, knows who you are, and merges per field

Promote and revert: through Git. That's what Git is good at

LINKS

GitOps Reverser
github.com/ConfigButler/gitops-reverser
API resources into Git as clean YAML

Reverse GitOps
reversegitops.dev
change through the API, a controller writes it to Git

The API comes first.
Git remembers.

IN GIT, ALL THIS TIME



30 / 240 seats claimed

But who changed it?

Two streams in. What changed, joined with who changed it.

The watch stream says what changed; the audit webhook says who. Each change waits only until its author is known, then joins a window. One window becomes one commit.

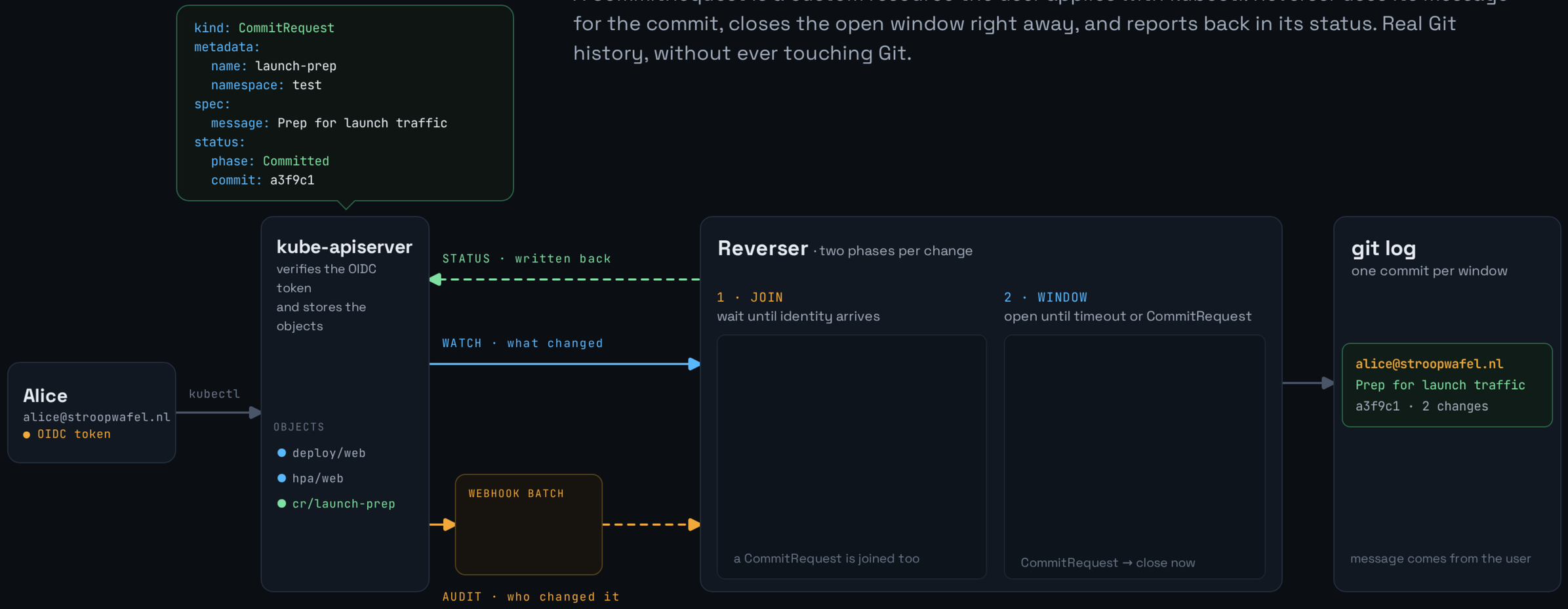


05 One commit by bob@stroopwafel.nl with all three: an update, a create and a delete.

01 change · 02 joined · 03 commit · 04 Bob: 3 changes · 05 commit

Say why, and the commit carries it.

A CommitRequest is a custom resource the user applies with kubectl. Reverser uses its message for the commit, closes the open window right away, and reports back in its status. Real Git history, without ever touching Git.



04 One commit by Alice with her reason as the message. The status now points to that commit.

01 changes · 02 CommitRequest · 03 window closed · 04 committed

Who wrote this commit? Two answers.

```yaml
apiVersion: configbutler.ai/v1alpha3
kind: GitProvider
metadata:
  name: k8s-trail
  namespace: voter
spec:
  url: ssh://git@github.com/ConfigButler/k8s-trail.git
  secretRef:
    name: k8s-trail-git
  commit:
    committer:
      name: ConfigButler Bot
      email: bot@configbutler.ai
    signing:
      secretRef:
        name: k8s-audit-trail-signing
```

Author

Whoever made the change. It comes from the audit log.

Committer

The bot that writes to Git. It's set right here.

Signature

Proves the bot wrote it. The key stays in a Secret, and the public key is in `status`.

Model it well,
and your files tell the story.

PART 4 OF 4

Let’s be fair: where doesn’t it fit?

1 **It’s just an API**
kube-apiserver and etcd, nothing else. One shape for API and file, and types you define.

2 **Identity is kept**
Your own login, for people and workloads, all the way to the commit.

3 **The API forgets, Git remembers**
GitOps for configuration: every change a commit, with who and why.

4 **It doesn’t always fit**
The downsides, and practical criteria for when to use it.

THE TEST I DISCUSSED YESTERDAY

A whole webshop catalogue, in KRM?

I think it fits

- Products and descriptions
- Categories and images
- Who changed this price?

It doesn't

- Stock
- Orders and payments
- Changes every second, or all or nothing

The catch: search

- No full-text search
- No joins, no sorting
- So copy it to a search index

The line runs through a single product.

PRACTICAL CRITERIA

Should you do this?

Good fit

- Small objects
- Changes now and then
- You need to know who changed it
- Permissions per type and per team

Bad fit

- Transaction data: orders, payments
- Rich queries and joins
- Changes across objects, all or nothing
- Big objects, or changes every second

Pros and cons

Pros

- People keep their settings screen; nobody needs to learn Git
- Structured, validated changes: schema, admission policies
- History with real authors, review and `git revert`
- The same API for people, scripts and AI agents, with the same limits
- See how settings are used: how often was something configured?
- You can move ownership: your settings are files in a repo
- Very flexible: policies and RBAC are resources too, so they get the same rigour

Cons

- Extra infrastructure: a config cluster plus Reverser
- Git runs 2 or 3 seconds behind
- Conflicts: incoming commits replay audit events, but if it comes to it, the API wins
- Mostly hidden, but people will recognize Kubernetes. `x-kubernetes-validations` even says it literally, unless you avoid CEL

LET'S BE FAIR

The kube-apiserver is ~~legacy~~ legendary.

Want the latest, as fast as you can?

Then run bleeding-edge Node.js.

Want it to evolve steadily, with great care?

A conservative project, and a large ecosystem around it.

Over a decade of production usage

It's not easy. It should be easy to play around with.

You

- Give access to a Git repo
- Create your CRDs

Me

- The kube-apiserver
- The login
- gitops-reverser

Would you try that? Come and tell me.

Is this contrarian?
And is there a chance it's right?

TLDR;

A control plane, just for your settings.

1. **It's just an API**
   kube-apiserver and etcd, nothing else. One shape for API and file, and types you define.

2. **Identity is kept**
   Your own login, for people and workloads, all the way to the commit.

3. **The API forgets, Git remembers**
   GitOps for configuration: every change a commit, with who and why.

4. **It doesn't always fit**
   The downsides, and practical criteria for when to use it.

And now, tell me what I've missed.

ONE ASK

What's the ingredient I'm missing?

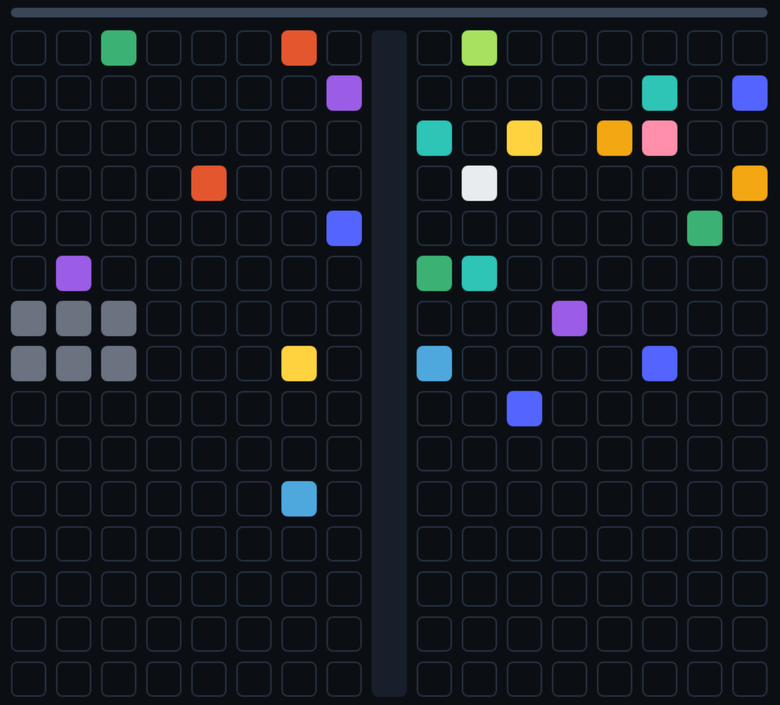
If this appeals somewhere: connect or let me know!

Tell me **one more ingredient**. Critical questions are the most welcome. I won't spam you. I'd just love to hear from you.

simon@koudijs.dev

The most valuable thing for me: people who give feedback.

THE ROOM · TURING



30 / 240 seats claimed

CONNECT · LINKEDIN

linkedin.com/in/simonkoudijs

FEEDBACK · SESSIONIZE

(Ab)using the Kubernetes API as a Configuration Backend

rate this session

koudijs.dev/speaking

The slides of both talks

Bi-directional GitOps github.com/ConfigButler/gitops-reverser/blob/main/docs/bi-directional.md

THE HARD PART OF BI-DIRECTIONAL GITOPS

Everything in the same order.

Flux on its own schedule: a race

Alice edits in the API → Her edit isn't in Git yet → Flux applies the older value → Her edit is gone

One writer per branch decides the moment

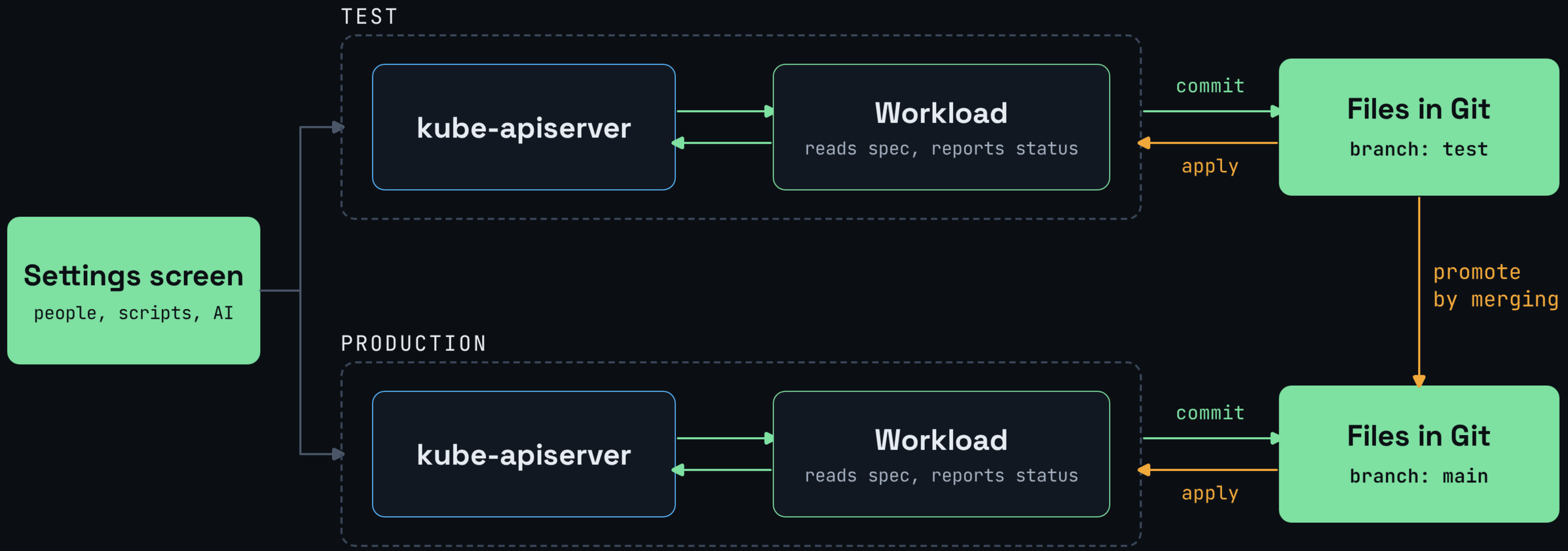
Alice edits in the API → Commit and push first → Then ask Flux to apply once → Her edit is safe

`FluxClaim` and `ArgoCDClaim` hold the applier: its own schedule is off, so it only applies when asked.

I got this wrong a few times. It's hard, and I'm open to improvements.

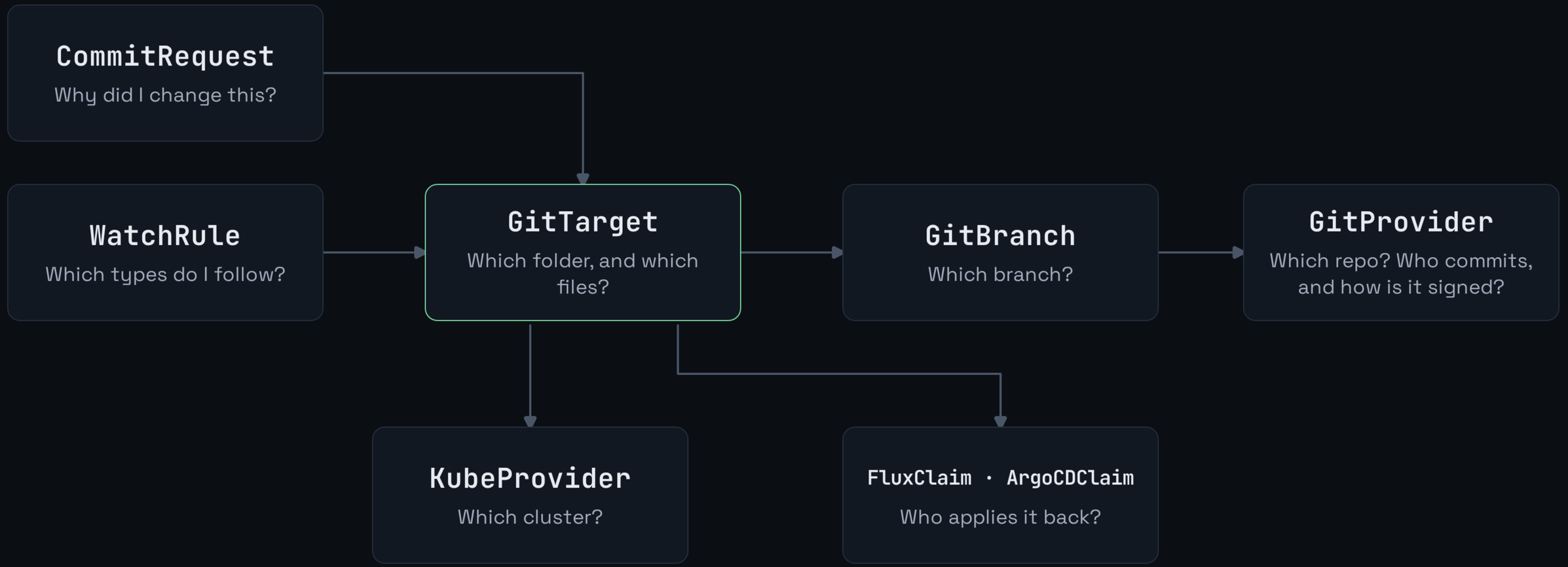
TLDR;

One block per stage. Git branches in between.



It goes both ways, so Git can promote.

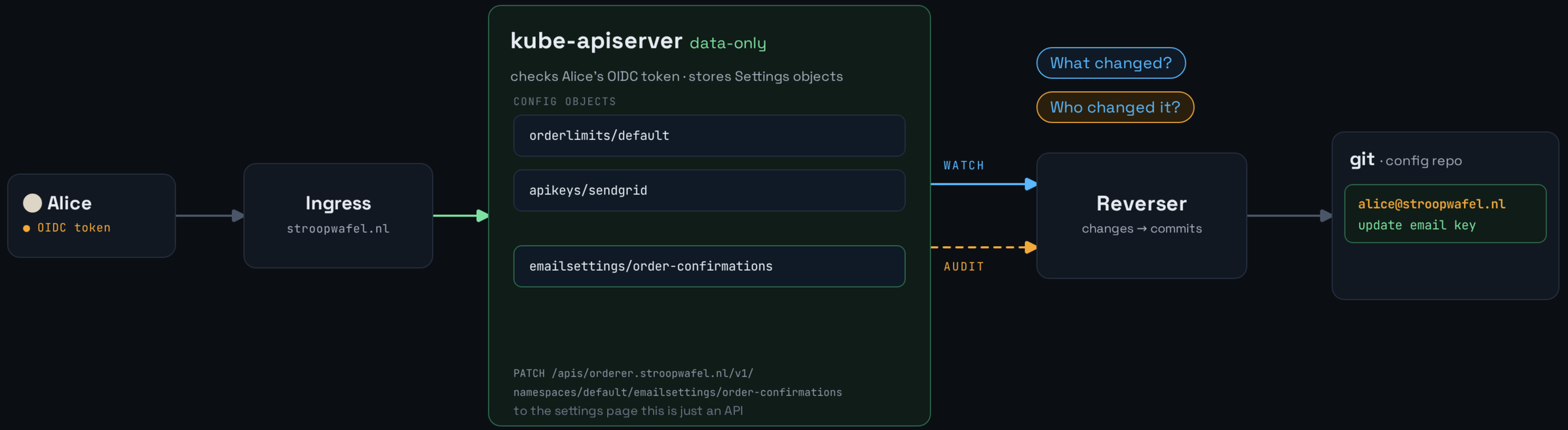
gitops-reverser is configured the same way.



Small objects that point at each other. Each one a kubectl get away.

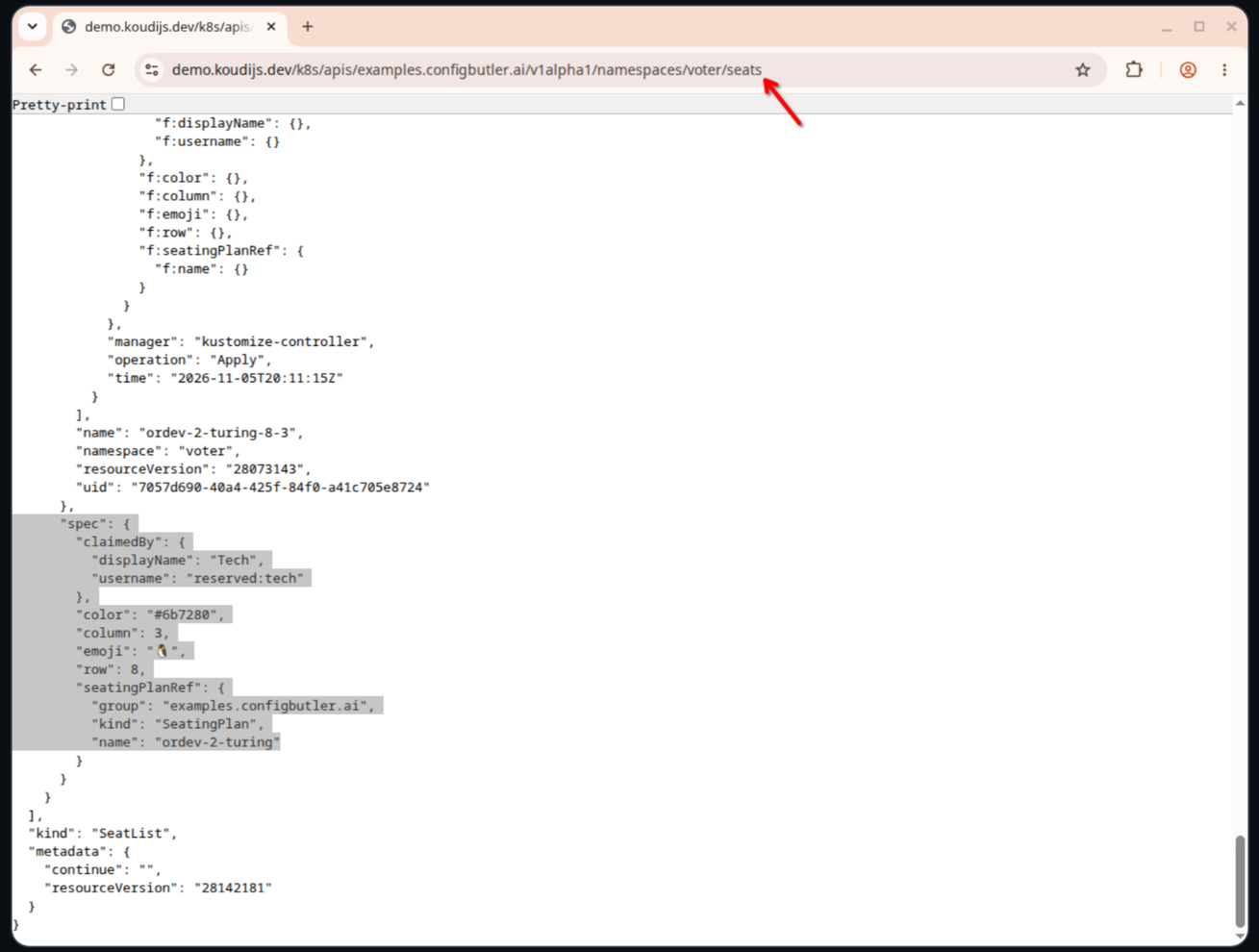
It’s just a kube-apiserver. Reverser listens.

Settings are Kubernetes objects. Every change goes through an apiserver that knows exactly who you are.



04 The apiserver already offers both, as two separate streams. Let’s look inside.

Leave off the name: every chair in the room.



- **The URL is the query.** One name less, and you get a list
- **Same shape, every type.** Nothing to learn for the next one
- **Spec is what you chose.** The rest is bookkeeping by the API server

LET'S DEFINE IT

Who is the API consumer?

Self-service platform

consumer — app teams

provider — platform engineers

"I need a database for my app."

SaaS product

consumer — customers

provider — SendGrid

"Configure the e-mail footer."

Both want to change something they don't run.

What do consumers expect these days?

Make it easy

- A screen I can just click
- An API for my scripts and AI agents
- My own login, with my own rights
- An instant answer if it's wrong

Leave me in control

- Who changed what, and why
- Undo, in one step
- My settings, in files I own

Not one or the other. Both.